



UÇTAN UCA KURULUM VE OPERASYON KILAVUZU
Grafana Anomaly Detector

Grafana Anomaly Detector v1.3.0 E2E Kılavuzu

Kurulumdan alarma, tüm datasource konfigürasyonlarından 7/24 feed mimarisine kadar eksiksiz teknik referans.

Plugin **v1.3.0**

Exporter **v1.3.0**

Grafana **11.6.7 / 12.4.0**

Python **3.9+**

15 Haziran 2026

NAVİGASYON

İçindekiler

<u>01 — Ne Yapar, Ne Yapmaz</u>	→
<u>02 — Sistem Gereksinimleri</u>	→
<u>03 — Mimari</u>	→
<u>04 — Kurulum</u>	→
<u>05 — İlk Konfigürasyon</u>	→
<u>06 — Panel Ekranları</u>	→
<u>07 — Algılama Algoritmaları</u>	→
<u>08 — Score Feed ve Hedef Seçimi</u>	→
<u>09 — 7/24 Feed (Panel Bağımsız)</u>	→
<u>10 — Tüm Butonlar</u>	→
<u>11 — Kaynak Datasource Kurulumu</u>	→
<u>12 — Sink Konfigürasyonu (Hedef Başına)</u>	→
<u>13 — Alarm Akışı — Baştan Sona</u>	→
<u>14 — Upgrade ve Rollback</u>	→
<u>15 — Doğrulama Komutları</u>	→
<u>16 — Troubleshooting</u>	→
<u>17 — Üretim Kontrol Listesi</u>	→

01

Ne Yapar, Ne Yapmaz

✓ Ne Yapar

- Grafana paneli içinde zaman serisi anomaly tespiti
- 0-100 normalize alert score üretimi
- 6 kaynak datasource tipi desteği (Prometheus, Loki, InfluxDB, PostgreSQL, ClickHouse, Elasticsearch)
- Hesaplanan score'u seçili hedefe yazar (6 hedef tipi)
- Panel kapalıyken dahi 7/24 sürekli feed (exporter eval loop)
- Hedef-farkında alarm sorgusu üretimi (PromQL / LogQL / Flux / SQL / DSL)
- Incident timeline, anomaly inspector, rule label önerileri

X Ne Yapmaz

- Bağımsız çalışmaz — Grafana gerektir
- Kendi veri deposu değil; veriyi dış store'lardan okur/yazar
- ML model eğitmez; istatistiksel algoritmalar kullanır (MAD, Z-score, Seasonal)
- Aynı score'u birden fazla hedefe yazmaz (tek hedef modeli)
- Alarm göndermez; Grafana Alerting'i tetikler
- Grafana Alerting'in yerine geçmez

Temel beklenti: Panel bir kez "Sync score feed" yapıldıktan sonra, panel kapatılsa da exporter kendi başına score hesaplar ve hedefe yazar. SOC 7/24 alarmı için panel açık tutmak *gerekmez*.

02

Sistem Gereksinimleri

BİLEŞEN	MİNİMUM	DOĞRULANAN	NOT
Grafana	10.0	11.6.7, 12.4.0	Panel API v2
Python	3.9	3.10, 3.11	Exporter için
RAM (exporter)	256 MB	512 MB önerilen	6 sink aktifse
Grafana rolü (alarm kurmak)	Editor	Editor	Alert rule, annotation için

BİLEŞEN	MİNİMUM	DOĞRULANAN	NOT
Ağ (push mode)	—	—	Tarayıcı → exporter :9110

03

Mimari

ADIM 1**Veri Okuma**

Panel veya Exporter, seçili kaynak datasource'tan zaman serisi alır.

ADIM 2**Anomaly Skoru**

MAD / Z-score / Seasonal → 0-100 alert score. Panel ve Exporter aynı matematiği kullanır.

ADIM 3**Hedefe Yaz**

Exporter eval loop (her 60 sn), score'u seçili hedefe yazar. Panel durumu önemsizdir.

ADIM 4**Grafana Alerting**

Hedef store her 3 dk sorgulanır. Score eşik üzerindeyse Pending başlar.

ADIM 5**Alarm Tetiklenir**

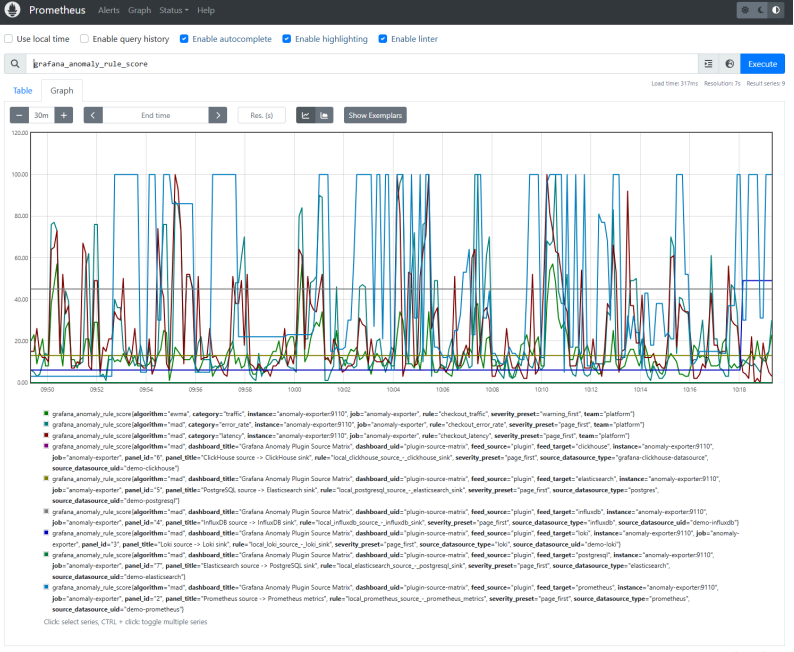
for: 10m dolduktan sonra Firing → Contact Point (e-posta, Slack, PagerDuty).

İki Yazma Kanalı

```
Panel (tarayıcı)
├─ POST /api/sync/panel → dynamic_rules.json (kalıcı kayıt)
├─ POST /api/feed/scores → anlık push (yalnız panel açıkken)

Exporter (Python servisi - 7/24)
├─ config.yml → statik kurallar
├─ dynamic_rules.json → panel-kaynaklı kalıcı kurallar
├─ run_once() her 60sn → kaynak oku → score hesapla → hedefe yaz
└─ ↑ panel açık/kapalı fark etmez
```

Çift yazma koruması: Bir kural `dynamic_rules.json` 'da kayıtlıysa, panel push geldiğinde exporter sink'e yazmaz — eval loop halleder. Kayıtlı değilse panel push doğrudan sink'e gider.



Exporter eval loop'tan gelen `grafana_anomaly_rule_score` zaman serisi — panel açık/kapalı fark etmeksizin her 60 saniyede güncellenir.

04

Kurulum

Plugin

```
cd /data/grafana/plugins
unzip /data/grafana/packages/grafana-anomaly-detector-plugin.zip
```

`grafana.ini` veya `custom.ini`:

```
[plugins]
allow_loading_unsigned_plugins = alpas-anomalydetector-panel
```

```
systemctl restart grafana-server
```

Hard refresh: Restart sonrası tarayıcıda Ctrl+Shift+R — eski JS bundle önbellekte kalabilir.

Exporter

```
mkdir -p /data/grafana/anomaly_exporter
cd /data/grafana/anomaly_exporter
unzip /data/grafana/packages/grafana-anomaly-exporter-bundle-1.3.0.zip

ANOMALY_PYTHON_BIN=$(command -v python3.9) \
ANOMALY_LISTEN_HOST=0.0.0.0 \
ANOMALY_LISTEN_PORT=9110 \
./portable-exporter.sh start http://127.0.0.1:8090
```

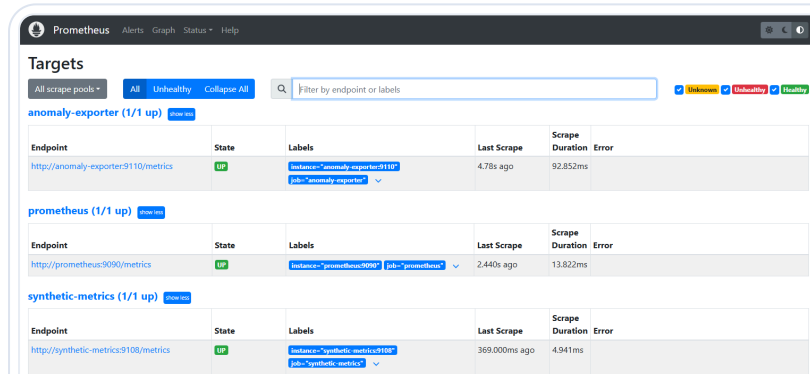
systemd Servisi (Opsiyonel)

```
cp grafana-anomaly-exporter.service /etc/systemd/system/
systemctl daemon-reload && systemctl enable --now grafana-anomaly-exporter
```

Kurulum Doğrulama

```
curl http://127.0.0.1:9110/health
# → {"status":"ok","version":"1.3.0", ... }

curl http://127.0.0.1:9110/metrics | grep grafana_anomaly_build_info
```



Prometheus					
Targets					
All scrape pools + All Unhealthy Collapse All Filter by endpoint or labels					
anomaly-exporter (1/1 up) View logs					
Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://anomaly-exporter:9110/metrics	UP	instance="anomaly-exporter:9110" job="anomaly-exporter"	4.78s ago	92.852ms	
prometheus (1/1 up) View logs					
Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://prometheus:9090/metrics	UP	instance="prometheus:9090" job="prometheus"	2.440s ago	13.822ms	
synthetic-metrics (1/1 up) View logs					
Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://synthetic-metrics:9108/metrics	UP	instance="synthetic-metrics:9108" job="synthetic-metrics"	369.000ms ago	4.941ms	

Prometheus Targets sayfası — grafana-anomaly-exporter UP, 9110 portundan scrape ediliyor.

05

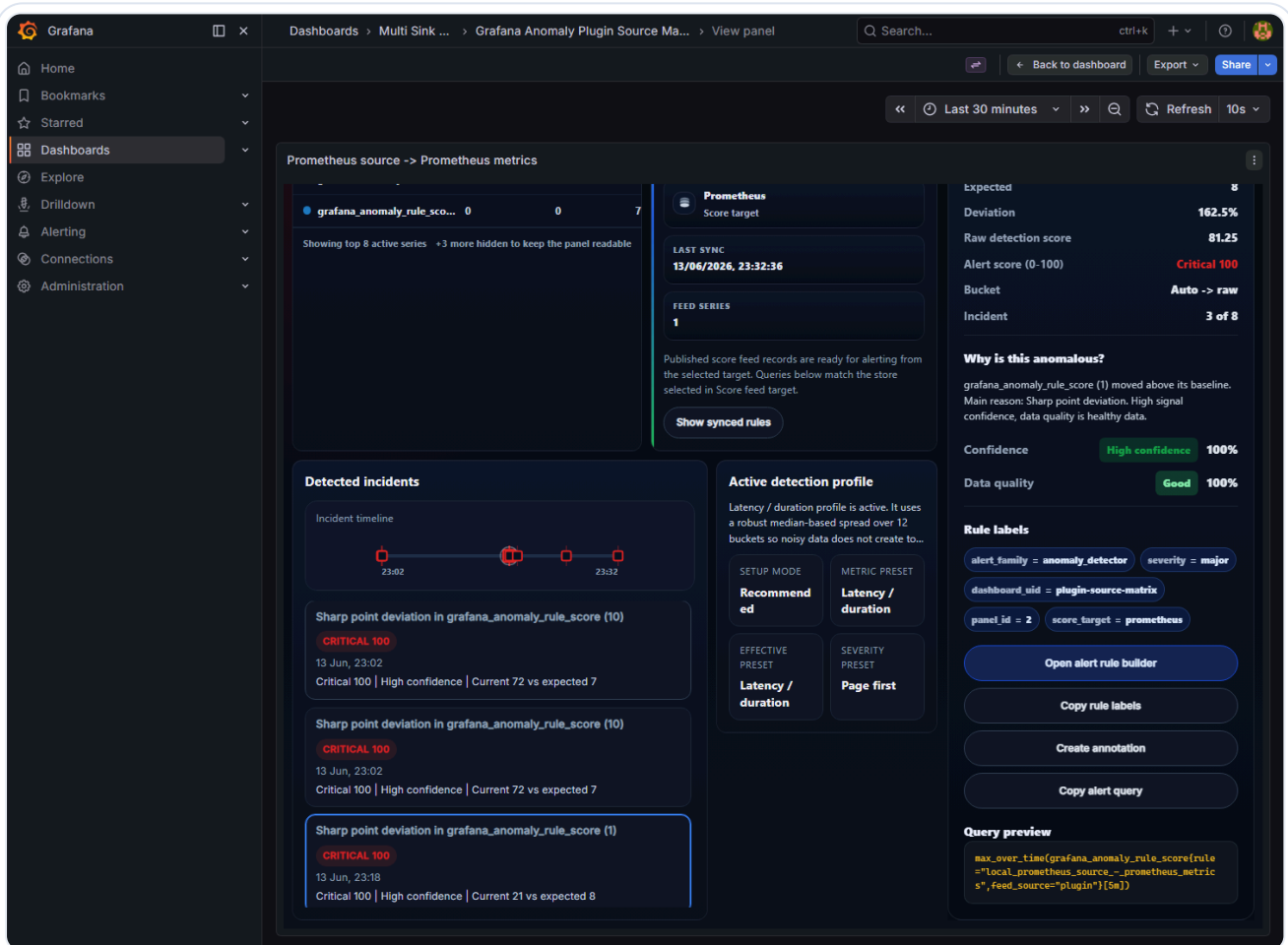
İlk Konfigürasyon

1. Grafana'da dashboard aç → **Panel ekle** → Visualization: *Anomaly Detector*
2. Query editor'da datasource seç, sorgu gir.
3. Panel Options → **Score feed target** seç (örn. *Prometheus*).
4. **Exporter endpoint:** `http://host:9110`
5. Apply — panel score hesaplamaya başlar.
6. Score feed bölümünden **Sync score feed**'e tıkla → ilk kayıt exporter'a gönderilir.
7. Doğrula: `curl http://host:9110/api/sync/rules`

Tek seferlik: Adım 6 yalnız bir kez yapılır. Bundan sonra panel kapalı olsa da exporter kendi başına score hesaplar ve hedefe yazar.

06

Panel Ekranları



Panel genel görünümü — SEVERITY Critical, CONFIDENCE 100%, ANOMALIES 404. Inspector bölmesi açık: raw/alert score, data quality, aksiyon butonları.

Prometheus source -> Prometheus metrics

Anomaly Detector

Algorithm: **mad**
Series: grafana_anomaly_rule_score (7)

Bucket: Auto -> raw
Datasource: Prometheus

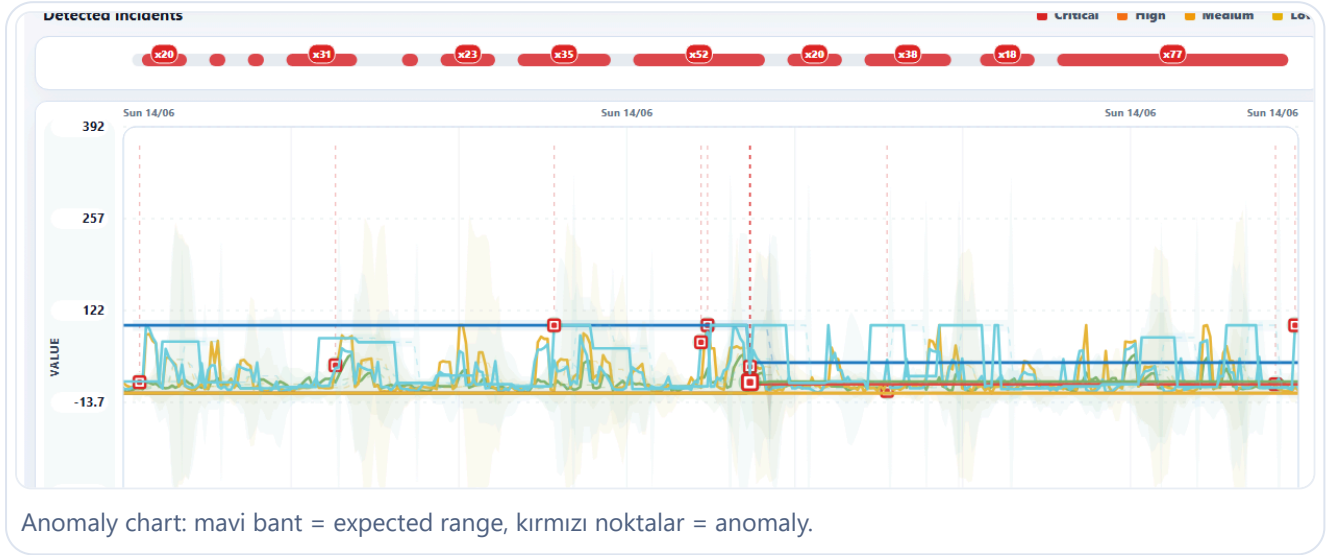
SEVERITY
Critical
Since 14/06 07:10

CONFIDENCE
100%
High confidence

ANOMALIES
336
clustered

DATA QUALITY
Good
100%

KPI header: SEVERITY, CONFIDENCE, ANOMALIES, DATA QUALITY ve seçili algoritma.



Anomaly score feed
Published 9 plugin-computed score series from the live panel to Prometheus metrics.

PROMETHEUS HEALTHY [Sync score feed](#)

STATUS
Synced

MODE
Auto sync

Prometheus
Score target

LAST SYNC
14/06/2026, 07:15:32

FEED SERIES
1

Published score feed records are ready for alerting from the selected target. Queries below match the store selected in Score feed target.

[Show synced rules](#)

[Open alert rule builder](#)

[Copy rule labels](#)

[Create annotation](#)

[Copy alert query](#)

Query preview

```
max_over_time(grafana_anomaly_rule_score{rule="local_prometheus_source_-_prometheus_metrics", feed_source="plugin"}[5m])
```

Score feed kartı: aktif hedef store, sync modu ve son sync zamanı.

Queries 1 **Transformations** 0

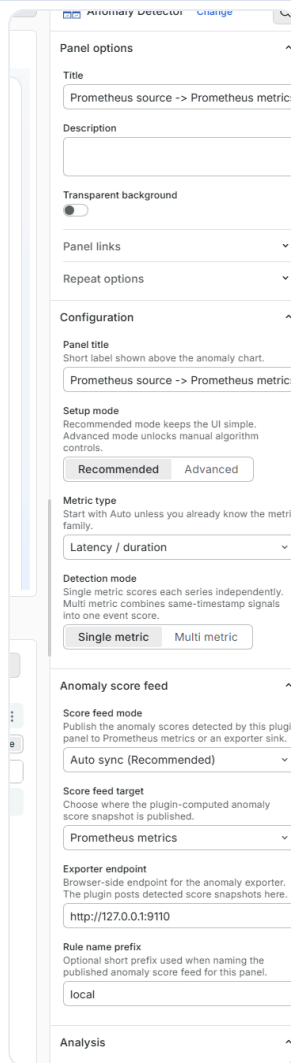
Data source: Prometheus [Query options](#) MD = auto = 1898 Interval = 5s [Query inspect](#)

A (Prometheus)

[Metrics browser](#) grafana_anomaly_rule_score

[Options](#) Legend: {{rule}} Format: Time series Step: auto Type: Range Exemplars: false

Query editor: kaynak datasource ve anomaly hesaplaması için sorgu tanımı.



The image shows the 'Anomaly Detector' configuration panel in Grafana. The panel is titled 'Anomaly Detector' and has a 'Change' button. It is divided into several sections: 'Panel options', 'Configuration', 'Anomaly score feed', and 'Analysis'. The 'Panel options' section includes fields for 'Title' (Prometheus source -> Prometheus metric), 'Description', and a 'Transparent background' toggle. The 'Configuration' section includes a 'Panel title' field, a 'Setup mode' selector (Recommended/Advanced), a 'Metric type' dropdown (Latency / duration), a 'Detection mode' selector (Single metric/Multi metric), an 'Anomaly score feed' section with a 'Score feed mode' dropdown (Auto sync (Recommended)), a 'Score feed target' dropdown (Prometheus metrics), an 'Exporter endpoint' field (http://127.0.0.1:9110), and a 'Rule name prefix' field (local). The 'Analysis' section is currently collapsed.

Panel options

Title
Prometheus source -> Prometheus metric

Description

Transparent background
☐

Panel links

Repeat options

Configuration

Panel title
Short label shown above the anomaly chart.
Prometheus source -> Prometheus metric

Setup mode
Recommended mode keeps the UI simple.
Advanced mode unlocks manual algorithm controls.
Recommended Advanced

Metric type
Start with Auto unless you already know the metric family.
Latency / duration

Detection mode
Single metric scores each series independently.
Multi metric combines same-timestamp signals into one event score.
Single metric Multi metric

Anomaly score feed

Score feed mode
Publish the anomaly scores detected by this plugin panel to Prometheus metrics or an exporter sink.
Auto sync (Recommended)

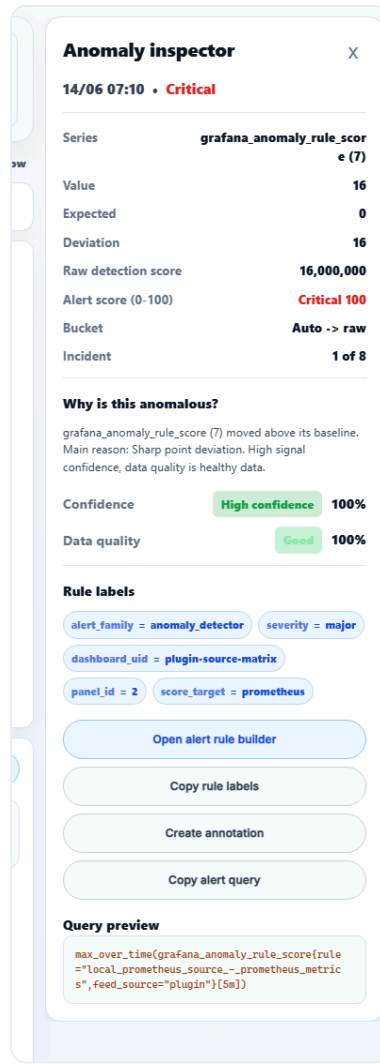
Score feed target
Choose where the plugin-computed anomaly score snapshot is published.
Prometheus metrics

Exporter endpoint
Browser-side endpoint for the anomaly exporter.
The plugin posts detected score snapshots here.
http://127.0.0.1:9110

Rule name prefix
Optional short prefix used when naming the published anomaly score feed for this panel.
local

Analysis

Panel options کنار çubuğu: setup mode, metric type, detection mode, score feed target, exporter endpoint.



Inspector aksiyon butonları: alert rule builder, label kopyala, annotation oluşturun, alert query kopyala.

Panel Ayarları Tablosu

AYAR	SEÇENEKLER	ÖNERİ
Setup mode	Recommended / Advanced	Recommended — güvenli default'lar
Metric type	Latency, Error rate, Throughput, Custom, Auto	Auto veya amaca uygun
Detection mode	Single / Multi	Single — birden fazla seri için Multi
Score feed target	Prometheus / Loki / InfluxDB / PostgreSQL / ClickHouse / Elasticsearch	Alarm altyapısıyla örtüşen
Score feed mode	Auto sync / Manual	Auto sync

AYAR	SEÇENEKLER	ÖNERİ
Exporter endpoint	URL	<code>http://host:9110</code>

07

Algılama Algoritmaları

MAD — Varsayılan

Median Absolute Deviation. Dağılım varsayımı yapmaz; aykırı değerlere karşı sağlam.

Ne zaman: Genel amaç, latency, error rate. Veri dağılımı bilinmiyorsa ilk tercih.

Z-score

Standart sapma tabanlı. Normal dağılımlı veri için uygundur.

Ne zaman: Veri yaklaşık Gaussian dağılımlıysa (örn. throughput ortalamaları).

Seasonal

Haftalık / günlük periyodik örüntüleri yakalar.

Ne zaman: Sabah–akşam veya hafta içi–sonu gibi düzenli döngüler olan metrikler.

Temel Parametreler

PARAMETRE	ARALIK	VARSAYILAN	ETKİSİ
Sensitivity (threshold)	1.0 – 10.0	4.0	Düşük → daha hassas (daha çok alarm). Yüksek → daha az alarm.
Baseline window	6 – 48	12	Beklenen değer hesabı için bakılan geçmiş periyot sayısı.
Seasonality samples	6 – 72	24	Yalnız Seasonal algoritmada geçerli; kaç periyot bakılacağını belirler.

Score paritesi: Panel (TypeScript `scoring.ts`) ve Exporter (Python `canonical.py`) aynı matematikle çalışır — aralarındaki fark $\leq 1e-6$. Panelde gördüğünüz score == hedefe yazılan score.

08

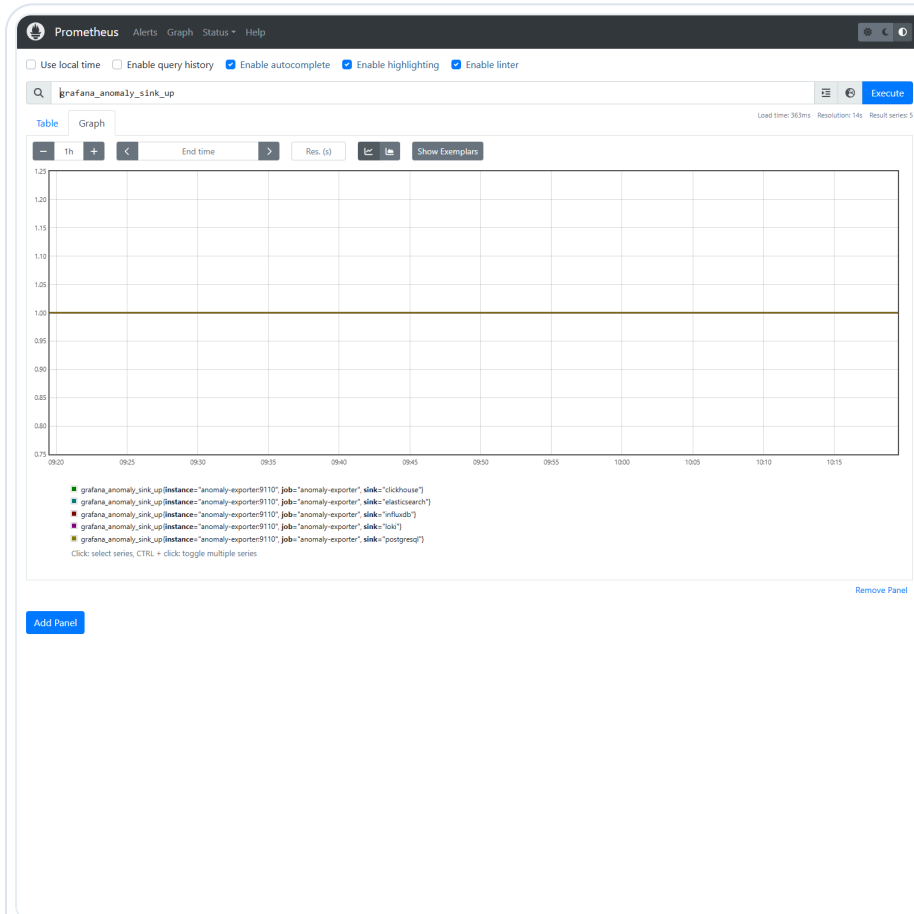
Score Feed ve Hedef Seçimi

Kaynak (veri nereden okunur) ile **hedef** (score nereye yazılır) birbirinden bağımsızdır. Örnek kombinasyonlar:

- PostgreSQL'den oku → Elasticsearch'e score yaz
- Prometheus'tan oku → Loki'ye score yaz
- Loki'den oku → PostgreSQL'e score yaz

Tüm 36 kombinasyon (6 kaynak × 6 hedef) doğrulanmıştır.

HEDEF	ALERT QUERY DİLİ	NE ZAMAN TERCİH EDİLİR
Prometheus metrics	PromQL	Standart Grafana alerting; Prometheus zaten altyapıda varsa
Loki	LogQL + unwrap	Log tabanlı alarm ekosistemi
InfluxDB	Flux	Time-series score depolama; Influx native alarm
PostgreSQL	SQL	Audit edilebilir relational kayıt
ClickHouse	SQL	Yüksek hacimli analitik
Elasticsearch	Query DSL	APM / log index ile entegrasyon



`grafana_anomaly_sink_up` : 5 sink'in tamamı `1.0` (sağlıklı). Birinin düşmesi → exporter log + config.yml kredensiyeli kontrol edilmeli.

7/24 Feed — Panel Bağımsız

Panel kapatılsa da feed durmuyor. Exporter `state/dynamic_rules.json` 'dan kuralı yeniden yükler ve eval loop'u kesintisiz sürdürür — hatta restart sonrasında bile.

Nasıl Çalışır

1. Panel → "Sync score feed" → `POST /api/sync/panel`
2. `DynamicRuleRegistry` kural kaydını `state/dynamic_rules.json` 'a yazar.
3. Exporter her eval döngüsünde: statik (`config.yml`) + dinamik (JSON) kuralları birleştir → kaynak oku → score hesapla → hedefe yaz. Panel durumu kontrol edilmez.
4. Restart sonrasında: `DynamicRuleRegistry.__init__` → `load()` → JSON'dan kurallar yüklenir → eval loop kaldığı yerden devam eder.

```
pretty-print [x]  
{  
  "rules": []  
}
```

`GET /api/sync/rules` yanıtı — panel sync sonrası oluşturulan dinamik kural kaydı. Panel kapalıyken de geçerlidir.

```
# Örnek /api/sync/rules yanıtı:  
{  
  "rules": [  
    {  
      "name": "checkout_latency",  
      "source_type": "prometheus",  
      "target_sink": "prometheus",  
      "algorithm": "mad",  
      "threshold": 4.0,  
      "registered_at": "2026-06-15T09:10:00Z",  
    }  
  ]  
}
```

```
"last_eval": "2026-06-15T09:11:00Z"
}
],
"dynamic_rule_count": 1
}
```

10

Tüm Butonlar

① Sync Score Feed

Panelde hesaplanan son score snapshot'ını exporter'a gönderir ve kural kaydını kalıcı hale getirir.

Akış	Panel → POST /api/sync/panel → dynamic_rules.json
Başarı	"Synced N rule(s)" mesajı; /api/sync/rules 'da kural görünür
Hata	Sync failed to fetch — exporter endpoint yanlış, CORS veya exporter down

② Show Synced Rules

Exporter'dan kayıtlı dinamik kuralları çekip listeler.

Akış	Panel → GET /api/sync/rules → liste modal
Başarı	Kural adı, kaynak, hedef, oluşturma zamanı görünür

③ Open Alert Rule Builder

Aktif score feed target'a göre doldurulmuş sorguyla Grafana alert builder'ı yeni sekmede açar.

Akış	Panel → URL parametreleri oluştur → window.open()
Başarı	Alert builder yeni sekmede; query ve condition ön doldurulmuş
Not	Grafana sürümüne göre bazı alanlar boş kalabilir — manuel olarak tamamla
Auth	Grafana Editor rolü gerekir

④ Copy Rule Labels

Routing ve silencing için önerilen minimal label setini panoya kopyalar.

Örnek	alert_family: anomaly_detector · rule: checkout_latency · feed_target: prometheus
Kullanım	Alert rule Labels alanına doğrudan yapıştır

11

Kaynak Datasource Kurulumu

Her kaynak için Grafana datasource ayarı ve Exporter global config örneği verilmiştir.

Prometheus

Grafana: Add data source → Prometheus → URL: `http://prometheus:9090`

Exporter `config.yml`:

```
global:
  prometheus_url: http://prometheus:9090
```

Panel query (PromQL): `rate(http_request_duration_seconds_count[5m])`

Loki

Grafana: Add data source → Loki → URL: `http://loki:3100`

```
global:
  loki_url: http://loki:3100
```

Panel query (LogQL metric): `sum(rate({app="myapp"}[5m]))`

InfluxDB

Grafana: Add data source → InfluxDB → Query language: **Flux** → URL: `http://influxdb:8086`

```
global:
  influxdb_url: http://influxdb:8086
  influxdb_token: my-token
  influxdb_org: myorg
```

Panel query (Flux):

```
from(bucket: "metrics")
  > range(start: -1h)
  > filter(fn: (r) => r._measurement == "cpu" and r._field == "usage")
  > aggregateWindow(every: 1m, fn: mean)
```

PostgreSQL

Grafana: Add data source → PostgreSQL → Host: `postgres:5432`

```
global:
  postgresql_dsn: postgresql://user:pass@postgres:5432/mydb
```

Panel query (SQL):

```
SELECT $__timeGroupAlias(ts, '1m'), avg(value) AS value
FROM metrics
WHERE $__timeFilter(ts) AND name = 'latency'
GROUP BY 1 ORDER BY 1
```

ClickHouse

Grafana: Add data source → ClickHouse → URL: `http://clickhouse:8123`

```
global:
  clickhouse_url: http://clickhouse:8123
  clickhouse_user: default
  clickhouse_password: ""
```

Panel query (SQL):

```
SELECT toStartOfMinute(ts) AS t, avg(value) AS value
FROM metrics
WHERE ts BETWEEN {start:DateTime} AND {end:DateTime}
GROUP BY t ORDER BY t
```

Elasticsearch

Grafana: Add data source → Elasticsearch → URL: `http://elasticsearch:9200`, index pattern girilir.

```
global:
  elasticsearch_url: http://elasticsearch:9200
  # Auth varsa:
  elasticsearch_username: elastic
  elasticsearch_password: changeme
```

12

Sink Konfigürasyonu — Hedef Başına

Exporter `config.yml` 'da `global` bölümü sink bağlantılarını, kurallar ise `target_sinks` ile hangi hedefe yazılacağını belirler.

Prometheus (ek sink gerekmez)

```
global:
  prometheus_url: http://prometheus:9090

rules:
  - name: checkout_latency
    source_type: prometheus
    query: histogram_quantile(0.95, sum(rate(checkout_duration_bucket[5m])) by (le))
    range_seconds: 3600
    step_seconds: 60
    algorithm: mad
```

```
threshold: 4.0
# target_sinks belirtilmez → score yalnız /metrics'te görünür
```

Loki Sink

```
global:
  prometheus_url: http://prometheus:9090
  loki_url: http://loki:3100 # zorunlu

rules:
  - name: checkout_latency
    source_type: prometheus
    query: ...
    algorithm: mad
    threshold: 4.0
    target_sinks: ["loki"]
```

InfluxDB Sink

```
global:
  prometheus_url: http://prometheus:9090
  influxdb_url: http://influxdb:8086 # zorunlu
  influxdb_token: my-influx-token # zorunlu
  influxdb_org: myorg # zorunlu
  influxdb_bucket: anomaly_scores # varsayılan: anomaly_scores

rules:
  - name: checkout_latency
    source_type: prometheus
    query: ...
    target_sinks: ["influxdb"]
```

PostgreSQL Sink

```
global:
  prometheus_url: http://prometheus:9090
  postgresql_dsn: postgresql://anomaly_user:pass@postgres:5432/anomaly_db

rules:
  - name: checkout_latency
    source_type: prometheus
    query: ...
    target_sinks: ["postgresql"]
```

ClickHouse Sink

```
global:
  prometheus_url: http://prometheus:9090
  clickhouse_url: http://clickhouse:8123
  clickhouse_user: default
  clickhouse_password: "" # auth yoksa boş

rules:
  - name: checkout_latency
    source_type: prometheus
```

```
query: ...  
target_sinks: ["clickhouse"]
```

Elasticsearch Sink

```
global:  
  prometheus_url: http://prometheus:9090  
  elasticsearch_url: http://elasticsearch:9200  
  elasticsearch_index: anomaly-scores # varsayılan: anomaly-scores  
  # Auth:  
  elasticsearch_username: elastic  
  elasticsearch_password: changeme  
  
rules:  
  - name: checkout_latency  
    source_type: prometheus  
    query: ...  
    target_sinks: ["elasticsearch"]
```

Çoklu Kural — Farklı Kaynak ve Hedefler

```
global:  
  prometheus_url: http://prometheus:9090  
  loki_url: http://loki:3100  
  postgresql_dsn: postgresql://user:pass@pg:5432/db  
  elasticsearch_url: http://es:9200  
  
rules:  
  - name: prom_to_loki          # Prometheus → Loki  
    source_type: prometheus  
    query: rate(http_requests_total[5m])  
    target_sinks: ["loki"]  
  
  - name: pg_to_elastic        # PostgreSQL → Elasticsearch  
    source_type: postgresql  
    query: "SELECT ts, value FROM metrics WHERE name='latency'"  
    target_sinks: ["elasticsearch"]  
  
  - name: prom_to_prometheus    # Prometheus → /metrics (varsayılan)  
    source_type: prometheus  
    query: up{job="api"}
```

/metrics çıktısı: grafana_anomaly_rule_score , grafana_anomaly_sink_up , build info ve eval cycle metrikleri.

Alarm Akışı — Baştan Sona

Tam Alarm Akışı

1

Datasource → Veri Okunur

Exporter eval loop, seçili kaynak datasource'tan zaman serisi alır (PromQL / LogQL / Flux / SQL / DSL).



2

Anomaly Score Hesaplanır

Python canonical.py scorer → 0-100 normalize alert score. Her eval döngüsünde, panel durumundan bağımsız.



3

Seçili Hedefe Yazılır

Prometheus: `/metrics` güncellenir (scrape ile alınır). Diğerleri: Loki, InfluxDB, PG, CH, ES'e doğrudan yazılır.



4

Grafana Alerting Sorgular (her 3 dk)

Alert rule, hedef store'u sorgular. Score > 90 →

Pending

başlar.



5

for: 10m Dolduktan Sonra → Firing

Skor 10 dk kesintisiz eşiğin üzerindeyse alarm

Firing

olur.



6

Contact Point Tetiklenir

Notification policy'ye göre: e-posta, PagerDuty, Slack, webhook.

Kritik: Alert rule konfigürasyonunda `noDataState=OK` ve `execErrState=OK` kullanmayın. Veri yokken veya sorgu hata verirken alarm yeşil kalır — sessiz körlük riski.

Alert Rule Ayarları

AYAR	DEĞER
Condition	<code>last(alert_score) > 90</code>
Evaluation interval	<code>3m</code>
For (pending süresi)	<code>10m</code>

AYAR	DEĞER
No data	NoData
Execution error	Error

Target Başına Alert Query

Prometheus

```
max_over_time(grafana_anomaly_rule_score{rule="checkout_latency"}[10m]) > 90
```

Loki

```
max_over_time({job="grafana_anomaly_exporter",rule="checkout_latency"}  
| json normalized_score="normalized_score"  
| unwrap normalized_score [10m]) > 90
```

InfluxDB (Flux)

```
from(bucket: "anomaly_scores")  
  > range(start: -10m)  
  > filter(fn: (r) => r.rule == "checkout_latency" and r._field == "normalized_score")  
  > last()
```

PostgreSQL / ClickHouse

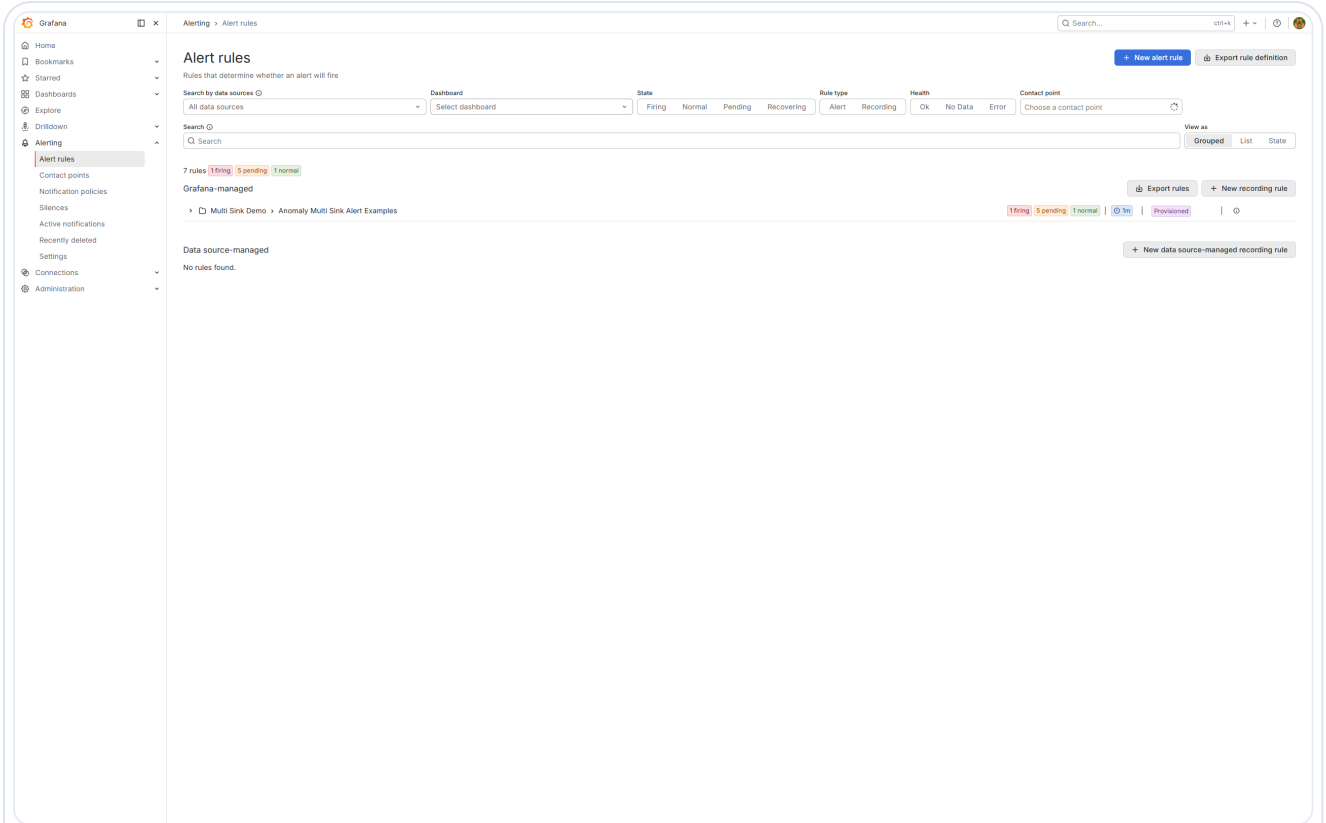
```
SELECT MAX(normalized_score) FROM anomaly_scores  
WHERE rule = 'checkout_latency' AND ts > NOW() - INTERVAL '10 minutes'
```

Elasticsearch

```
{"query":{"bool":{"filter":[{"term":{"rule":"checkout_latency"}},  
{"range":{"@timestamp":{"gte":"now-10m"}}}]}},  
"aggs":{"max_score":{"max":{"field":"normalized_score"}}}}
```

Watchdog Alarmları

```
up{job="grafana-anomaly-exporter"} == 0  
time() - grafana_anomaly_sink_last_write_timestamp_seconds > 300  
grafana_anomaly_sink_up == 0
```



Grafana Alert rules listesi: rule grupları, firing/normal durum ve son değerlendirme zamanları.

14

Upgrade ve Rollback

Yedek Al

```
TS=$(date +%Y%m%d_%H%M%S)
mkdir -p /data/grafana/backup/anomaly_$TS
cp -a /data/grafana/plugins/alpas-anomalydetector-panel /data/grafana/backup/anomaly_$TS/plugin
cp -a /data/grafana/anomaly_exporter /data/grafana/backup/anomaly_$TS/exporter
```

Plugin Upgrade

```
systemctl stop grafana-server
mv /data/grafana/plugins/alpas-anomalydetector-panel{,.old.$TS}
unzip /data/grafana/packages/grafana-anomaly-detector-plugin.zip -d /data/grafana/plugins
systemctl start grafana-server
```

Exporter Upgrade

```
./portable-exporter.sh stop
mv /data/grafana/anomaly_exporter{,.old.$TS}
mkdir /data/grafana/anomaly_exporter && cd /data/grafana/anomaly_exporter
unzip /data/grafana/packages/grafana-anomaly-exporter-bundle-1.3.0.zip
```

```
cp /data/grafana/backup/anomaly_${TS}/exporter/state/dynamic_rules.json state/  
./portable-exporter.sh start http://127.0.0.1:8090
```

Rollback

```
# Plugin:  
systemctl stop grafana-server  
rm -rf /data/grafana/plugins/alpas-anomalydetector-panel  
mv /data/grafana/plugins/alpas-anomalydetector-panel{.old.${TS},}  
systemctl start grafana-server  
  
# Exporter:  
./portable-exporter.sh stop  
rm -rf /data/grafana/anomaly_exporter  
mv /data/grafana/anomaly_exporter{.old.${TS},}  
cd /data/grafana/anomaly_exporter && ./portable-exporter.sh start http://127.0.0.1:8090
```

15

Doğrulama Komutları

Sağlık

```
curl http://host:9110/health
curl http://host:3000/api/health
curl http://host:8090/-/healthy
```

Metrik

```
curl http://host:9110/metrics \
| grep -E 'rule_score|sink_up|build_info'
```

Score Parity

```
python3 scripts/parity_check.py
# Beklenen: panel_score=N fed_score=N
```

Panel Feed Continuity

```
# Panel kapat → 25 sn bekle →
curl http://host:9110/metrics \
| grep last_write_timestamp
# Değer ilerlemiş olmalı
```

SHA256 Paket Doğrulama

```
sha256sum -c SHA256SUMS_v1.3.0.txt # 6/6 OK beklenir
```

The screenshot shows the Grafana Prometheus interface with the 'grafana_anomaly_rule_score' table view. The table contains the following data:

Algorithm	Category	Instance	Job	Rule	Severity Preset	Team	Score
grafana_anomaly_rule_score[algorithm="mad", category="latency", instance="anomaly-exporter9110", job="anomaly-exporter", rule="checkout_latency", severity_preset="page_first", team="platform"]	latency	anomaly-exporter9110	anomaly-exporter	checkout_latency	page_first	platform	9
grafana_anomaly_rule_score[algorithm="ewma", category="traffic", instance="anomaly-exporter9110", job="anomaly-exporter", rule="checkout_traffic", severity_preset="warning_first", team="platform"]	traffic	anomaly-exporter9110	anomaly-exporter	checkout_traffic	warning_first	platform	10
grafana_anomaly_rule_score[algorithm="mad", category="error_rate", instance="anomaly-exporter9110", job="anomaly-exporter", rule="checkout_error_rate", severity_preset="page_first", team="platform"]	error_rate	anomaly-exporter9110	anomaly-exporter	checkout_error_rate	page_first	platform	34
grafana_anomaly_rule_score[algorithm="mad", dashboard_title="Grafana Anomaly Plugin Source Matrix", dashboard_uid="plugin-source-matrix", feed_source="plugin", feed_target="loki", instance="anomaly-exporter9110", job="anomaly-exporter", panel_id="3", panel_title="Loki sink", rule="local_loki_source -> Loki sink", severity_preset="page_first", source_datasource_type="loki", source_datasource_uid="demo-loki"]	mad	Grafana Anomaly Plugin Source Matrix	plugin-source-matrix	plugin	loki	anomaly-exporter9110	49
grafana_anomaly_rule_score[algorithm="mad", dashboard_title="Grafana Anomaly Plugin Source Matrix", dashboard_uid="plugin-source-matrix", feed_source="plugin", feed_target="prometheus", instance="anomaly-exporter9110", job="anomaly-exporter", panel_id="2", panel_title="Prometheus source -> Prometheus metrics", rule="local_prometheus_source -> Prometheus metrics", severity_preset="page_first", source_datasource_type="prometheus", source_datasource_uid="demo-prometheus"]	mad	Grafana Anomaly Plugin Source Matrix	plugin-source-matrix	plugin	prometheus	anomaly-exporter9110	100
grafana_anomaly_rule_score[algorithm="mad", dashboard_title="Grafana Anomaly Plugin Source Matrix", dashboard_uid="plugin-source-matrix", feed_source="plugin", feed_target="influxdb", instance="anomaly-exporter9110", job="anomaly-exporter", panel_id="4", panel_title="InfluxDB source -> InfluxDB sink", rule="local_influxdb_source -> influxdb_sink", severity_preset="page_first", source_datasource_type="influxdb", source_datasource_uid="demo-influxdb"]	mad	Grafana Anomaly Plugin Source Matrix	plugin-source-matrix	plugin	influxdb	anomaly-exporter9110	45
grafana_anomaly_rule_score[algorithm="mad", dashboard_title="Grafana Anomaly Plugin Source Matrix", dashboard_uid="plugin-source-matrix", feed_source="plugin", feed_target="elasticsearch", instance="anomaly-exporter9110", job="anomaly-exporter", panel_id="5", panel_title="PostgreSQL source -> Elasticsearch sink", rule="local_postgresql_source -> elasticsearch_sink", severity_preset="page_first", source_datasource_type="postgres", source_datasource_uid="demo-postgresql"]	mad	Grafana Anomaly Plugin Source Matrix	plugin-source-matrix	plugin	elasticsearch	anomaly-exporter9110	13
grafana_anomaly_rule_score[algorithm="mad", dashboard_title="Grafana Anomaly Plugin Source Matrix", dashboard_uid="plugin-source-matrix", feed_source="plugin", feed_target="postgresql", instance="anomaly-exporter9110", job="anomaly-exporter", panel_id="6", panel_title="Elasticsearch source -> PostgreSQL sink", rule="local_elasticsearch_source -> postgresql_sink", severity_preset="page_first", source_datasource_type="elasticsearch", source_datasource_uid="demo-elasticsearch"]	mad	Grafana Anomaly Plugin Source Matrix	plugin-source-matrix	plugin	postgresql	anomaly-exporter9110	0
grafana_anomaly_rule_score[algorithm="mad", dashboard_title="Grafana Anomaly Plugin Source Matrix", dashboard_uid="plugin-source-matrix", feed_source="plugin", feed_target="clickhouse", instance="anomaly-exporter9110", job="anomaly-exporter", panel_id="7", panel_title="ClickHouse source -> ClickHouse sink", rule="local_clickhouse_source -> clickhouse_sink", severity_preset="page_first", source_datasource_type="grafana-clickhouse-datasource", source_datasource_uid="demo-clickhouse"]	mad	Grafana Anomaly Plugin Source Matrix	plugin-source-matrix	plugin	clickhouse	anomaly-exporter9110	0

Grafana table view — kural adı, son normalize score ve timestamp. Her eval döngüsünde (60 sn) güncellenir.

16

Troubleshooting

BELİRTİ	NEDEN	ÇÖZÜM
Sync failed to fetch	Exporter down, CORS, proxy	<code>curl host:9110/health</code> ; endpoint düzelt; CORS başlığı ekle
Panel score yok	Query boş döndürüyor; zaman aralığı veri içermiyor	Query editor'da test et; zaman aralığını genişlet
dynamic_rule_count=0	state/dynamic_rules.json eksik veya izin sorunu	<code>ls -la state/</code> ; izinleri düzelt; yeniden sync et
Alert builder boş açılıyor	Grafana sürüm uyumsuzluğu	Copy alert query + Copy rule labels ile manuel tamamla
sink_up=0	Hedef store down; credentials değişti	Sink servisini kontrol et; config.yml'ı güncelle; exporter restart
ES'de yazım gözükmüyor	<code>terms</code> agg text alanda — fielddata kapalı	<code>rule.keyword</code> ile sorgula; ham hit sayısını kontrol et

BELİRTİ	NEDEN	ÇÖZÜM
NoData mailleri herkese gidiyor	Default notification policy tüm alertleri yakalıyor	<code>alert_family=anomaly_detector</code> label ile ayrı policy oluştur

Log Erişimi

```
tail -f /data/grafana/anomaly_exporter/logs/exporter.log
journalctl -u grafana-anomaly-exporter -f
docker logs -f anomaly_exporter --tail 100
```

Üretim Kontrol Listesi

Kurulum

- ☐ Plugin v1.3.0 kuruldu ve Grafana tanıyor
- ☐ Exporter v1.3.0 çalışıyor, health 200 OK
- ☐ SHA256SUMS 6/6 OK
- ☐ Yedek alındı (plugin + exporter + state)

Konfigürasyon

- ☐ config.yml — kullanılan sink URL/credential'ları girildi
- ☐ Panel score feed target doğru seçili
- ☐ Panel exporter endpoint girili ve erişilebilir
- ☐ Datasource bağlantısı Grafana'da test edildi
- ☐ Tüm sink'ler sink_up=1

Feed ve 7/24

- ☐ En az bir panel sync yapıldı
- ☐ state/dynamic_rules.json yazılabilir dizinde mevcut
- ☐ Panel kapalıyken sink timestamp'leri ilerledi (5 dk beklendi)
- ☐ Exporter restart sonrası dynamic_rule_count doğru

Alarm

- ☐ Alert rule condition: 0-100 alert score üzerinden (> 90)
- ☐ noDataState ≠ OK; execErrState ≠ OK
- ☐ for ≥ 10m
- ☐ Contact point test bildirimi alındı
- ☐ Watchdog alarm (exporter_up, sink_up, yazma gecikmesi) kuruldu

Grafana Anomaly Detector v1.3.0 — E2E Kılavuzu — 15 Haziran 2026 · Uygulama koduna dokunulmadı.